

Resoluciones de ejercicios 1, 2 y 3

Top de IP's por Cantidad de Requests

```
cut "$1" -d ' ' -f 1 | sort | uniq -c | sort -nr
```

- Utilizamos el comando `cut` para aislar las IP's.
- Luego, usamos el comando `sort` para que las IP's repetidas queden contiguas, para luego poder utilizar el comando `uniq` y contar las veces que una IP se repite.
- Finalmente, usamos nuevamente el comando `sort` para ordenar los resultados de mayor a menor a partir de un orden numérico.

Requests Fallidas

```
cut -d ' ' -f 1,6,7,8,9 "$1" | grep -E " 4[[:digit:]]{2}" | tr ' ' '\t'
```

- Como sabemos que los valores de cada log no cambian su posición, usamos `cut` con valores fijos para las columnas que necesitamos.
- Luego filtramos aquellas líneas en las que el código de salida tiene la forma 4xx usando `grep`. Notar que el espacio inicial es intencional para ser preciso con el inicio del código de salida.
- Luego, como los recursos están encerrados entre comillas, intercambiamos esas comillas por `'\t'` para que los datos queden separados por tabuladores usando `tr`.

Recursos más Solicitados

```
grep "\"GET \" \"$1" | cut -d ' ' -f 7 | sort | uniq -c | sort -nr | head -n 3
```

- Filtramos con `grep` los logs que hayan utilizado el método GET.
- Al igual que antes, usamos `cut` con valores fijos para aislar los path utilizados.
- Finalmente, utilizamos un pipeline como en el ejercicio 1 para ordenar los resultados y utilizamos el comando `head` para imprimir el top 3.

Version Generalizada

```
read -p "Ingrese el método a buscar: " var && grep "\"$var \" \"$1" | cut -d ' ' -f 7 | sort |  
uniq -c | sort -nr | head -n 3
```

La diferencia está en que al inicio se pide un input al usuario para que ingrese un método. Esto hace que el pipeline esté generalizado.

Script de Reporte

```
#!/bin/bash

if [ ! -e $1 ]; then
    echo "archivo no encontrado" && exit 1
fi

ip_list=$(cut -d ' ' -f 1 $1 | sort | uniq)

echo "== reporte de $1 =="

for ip in $ip_list; do
    ip_requests=$(grep -w "$ip" "$1")
    ip_total_requests=$(echo "$ip_requests" | wc -l)
    ip_errors=$(echo "$ip_requests" | cut -d ' ' -f 9 | grep -E 4[[:digit:]]{2} | wc -l)

    echo -e "ip: $ip\t\trequests: $ip_total_requests\t\tfallidos: $ip_errors"
done
```

- Verificamos si el archivo existe. Si no es así, el programa tendrá un `exit` de error 1.
- `ip_list` contiene la lista de todas las IP's en el log. Se consigue aplicando una lógica similar al ejercicio 1.
- Se itera sobre cada IP usando un `FOR` en el que en cada iteración se filtran las líneas en la que aparece la IP iterado para guardar los resultados en `'ip_request'`. `'ip_total_requests'` contiene el número de líneas que contiene `'ip_requests'` (número de apariciones de la IP). Se utiliza el comando `echo` antes de usar `wc` para que este último funcione correctamente. Luego `'ip_errors'` contiene el número de veces que las requests de la IP devuelven un código de error, utilizando una lógica similar al ejercicio 2. El espacio al inicio en el regex no es necesario porque solo trabajamos con la columna donde están los códigos de salida.